

System Architecture & Design

Task 1 — Service Marketplace, Product Marketplace & Bidding Platform

Prepared by: Gayu — Full-Stack Developer
Assessment: Technical Aptitude Assessment (Task 1 of 2)
Date: 13 July 2026
Contents: Architecture, Database Schema, API Structure, Auth Strategy, ER Diagram

0. Approach & Assumptions

This document covers the system design for a platform with three core modules — a service marketplace (listings, orders, escrow), a product marketplace (catalog, cart, inventory), and a bidding system (project requests, competitive bids) — all sharing a common authentication, messaging and notification layer. I've designed this as I would a real production system: a modular monolith at the API layer with clearly separated domain services, backed by a single relational database (with room to split into microservices later if traffic demands it). A few assumptions I made explicit up front:

- Roles are not mutually exclusive — the same user account can be a Client in Module 1 and a Buyer in Module 2. Role is therefore modelled per-context rather than as a single fixed field where it matters (see §4.3).
- Module 1's escrow/order logic is reused by Module 3 when a bid is accepted, as the spec explicitly asks for — this drove me to design a single shared orders table with a polymorphic `source_type` rather than three separate order tables.
- I've optimised this write-up for clarity over exhaustiveness — every table/endpoint that matters for the "what you have to do" checklist is here, but I haven't listed every CRUD permutation.

1. System Architecture

1.1 High-level overview

I went with a **modular monolith** for the API layer rather than full microservices. At this stage the three modules share too much (auth, users, messaging, and — critically — the escrow/order logic that Module 3 reuses from Module 1) for splitting them into separately-deployed services to pay off. Instead, I've drawn hard boundaries *inside* the codebase (separate modules/packages, no cross-module DB queries except through service interfaces) so that if one module later needs to scale or be extracted (Module 2's inventory locking is the most likely candidate, given concurrency pressure), it can be pulled out without a rewrite. The one piece I did split out physically from day one is background/async work — notifications, bid expiry, and anything that shouldn't block a request — because that's cheap to do early and painful to retrofit.

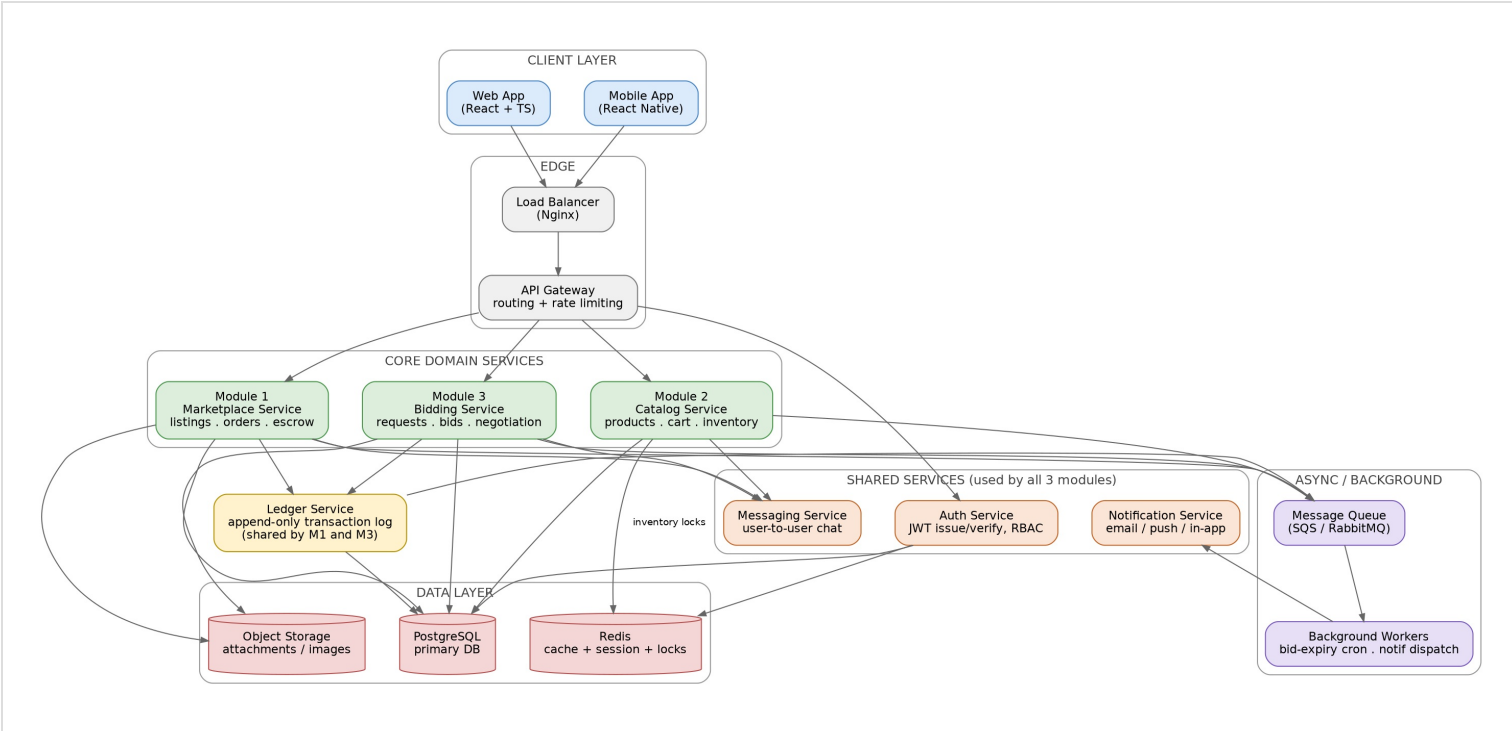


Fig. 1 — System architecture (component diagram)

1.2 Why this shape

- **API Gateway in front of everything** — single entry point for auth checks, rate limiting (important for Module 2's search/filter endpoints and Module 3's bid submission, both of which are easy to hammer), and request logging.
- **Auth Service is shared, not duplicated** — issues and verifies JWTs, exposes a `/me` and role-check middleware that all three domain services call. This is what the spec means by "modules share authentication."
- **Ledger Service sits between Module 1/3 and the DB** — rather than letting each module write ledger rows directly, all fund-custody state changes go through one component. This is where I enforce "every money-state transition is atomic and auditable" (§2 requirement) in one place instead of three.
- **Queue + workers for anything async** — notifications, bid-expiry checks, and (later) email/SMS dispatch. Keeps the request/response path fast and means a notification failure never blocks an order action.
- **Redis** is used for two different things and I kept that distinction explicit in my notes: session/ cache, and short-lived locks for Module 2's inventory decrement (a Redis `SETNX`-based lock or Postgres row-level locking — I go with Postgres row locks in §2.2 since it keeps the source of truth in one place, and only reach for Redis locking if profiling shows the DB is the bottleneck).

1.3 Deployment

- **Containers:** each service (API, workers, gateway) is a separate Docker image, orchestrated with Docker Compose for local dev and ECS/Kubernetes for staging+prod. Keeping them as separate images now is what makes an eventual microservice split cheap later, even though they currently deploy together as one release.
- **CI/CD:** GitHub Actions — on PR: lint, unit tests, concurrency stress tests (mandatory per Task 2's spec, but I'd apply the same discipline here); on merge to main: build image, push to registry, deploy to staging automatically, prod on manual approval.
- **Database:** managed Postgres (RDS/Cloud SQL) with automated backups and a read replica once read traffic (search/filter/browse) grows enough to warrant separating reads from writes.
- **Static assets/attachments:** S3-compatible object storage, served via CDN, not through the app servers.

2. Database Schema

I'm using a single relational database (PostgreSQL) shared across modules with foreign keys enforcing integrity where modules connect (e.g. an order pointing back to whichever module created it). Below are the core tables grouped by module, followed by the shared tables.

2.1 Shared / cross-module tables

Table	Column	Type	Notes
users	id	uuid	PK
	name	varchar	
	email	varchar	unique, indexed
	password_hash	varchar	bcrypt/argon2
	role	enum	base role for admin panel access; per-context role is inferred from which table references the user (see §4.3)
	created_at	timestamp	
notifications	id	uuid	PK
	user_id	uuid	FK → users.id
	type	varchar	order_accepted, bid_received, dispute_opened, ...
	payload	jsonb	reference ids + display data
	read_at	timestamp	nullable
	created_at	timestamp	
messages	id	uuid	PK
	thread_id	uuid	FK → message_threads.id
	sender_id	uuid	FK → users.id
	body	text	
	created_at	timestamp	

2.2 Module 1 — Service Marketplace

Table	Column	Type	Notes
listings	id	uuid	PK
	provider_id	uuid	FK → users.id
	title, description	varchar/text	
	price	decimal(12,2)	
	category	varchar	indexed for filter
	status	enum	active / inactive

Design note — why orders is shared across Module 1 & 3: Instead of a separate orders table per module, I use one table with a polymorphic source_type + source_id pair (points at either a listing or an accepted bid). This is the "reuse Module 1's order/escrow logic" bonus point in the spec, done at the schema level rather than by duplicating the state machine in Module 3's code.

Table	Column	Type	Notes
orders	id	uuid	PK
	source_type	enum	'listing' 'bid'
	source_id	uuid	FK → listings.id or bids.id (app-level, not a DB FK, since it's polymorphic)
	client_id	uuid	FK → users.id
	provider_id	uuid	FK → users.id
	amount	decimal(12,2)	
	status	enum	pending → accepted/declined → funded → held → delivered → completed disputed
	version	int	optimistic lock – incremented on every status change
	created_at, updated_at	timestamp	
ledger_entries	id	uuid	PK
	order_id	uuid	FK → orders.id, indexed
	actor_id	uuid	FK → users.id (who triggered it)
	action	enum	fund / hold / release / dispute / resolve
	amount	decimal(12,2)	
	state_before	varchar	order status before this entry

	state_after	varchar	order status after this entry
	created_at	timestamp	append-only, never updated
disputes	id	uuid	PK
	order_id	uuid	FK → orders.id
	raised_by	uuid	FK → users.id
	resolved_by	uuid	FK → users.id, nullable (admin)
	reason	text	
	resolution	enum	released_to_provider / refunded_to_client / pending
	created_at, resolved_at	timestamp	

On "atomic and auditable": every write to orders.status happens inside a DB transaction that also inserts the matching ledger_entries row and bumps orders.version. Nothing writes to the ledger without touching the order in the same transaction, and nothing changes order status without a ledger row — so the ledger is a complete, replayable history, not a side log that can drift.

2.3 Module 2 — Product Marketplace

Table	Column	Type	Notes
products	id	uuid	PK
	vendor_id	uuid	FK → users.id
	sku	varchar	unique per vendor
	name, description	varchar/text	
	unit_price	decimal(12,2)	
	unit_of_measure	varchar	box / unit / ton
	available_qty	int	decremented atomically on order (see §2.3 note)
	version	int	optimistic lock
carts / cart_items	id	uuid	PK
	buyer_id	uuid	FK → users.id
	product_id	uuid	FK → products.id
	quantity	int	
order_items	id	uuid	PK
	order_id	uuid	FK → orders.id
	product_id	uuid	FK → products.id
	quantity, unit_price	int/decimal	price snapshotted at order time
	fulfillment_status	enum	pending / partial / fulfilled / backordered

Design note — preventing overselling: I decrement available_qty inside the same transaction as the order-item insert, using UPDATE products SET available_qty = available_qty - :qty, version = version + 1 WHERE id = :id AND available_qty >= :qty. If the row count returned is 0, the request fails with a 409 and the client retries or the order is rejected — this is what makes the decrement safe under concurrent requests without needing an explicit application-level lock. Multi-item orders wrap all product decrements in a single DB transaction so a partial failure never leaves inventory in a half-decremented state.

2.4 Module 3 — Bidding System

Table	Column	Type	Notes
project_requests	id	uuid	PK
	requester_id	uuid	FK → users.id
	description	text	
	budget_min, budget_max	decimal(12,2)	
	deadline	timestamp	bids auto-expire after this
	status	enum	open / closed / expired
bids	id	uuid	PK
	request_id	uuid	FK → project_requests.id
	bidder_id	uuid	FK → users.id
	price	decimal(12,2)	
	message	text	
	estimated_completion	date	

	status	enum	pending / accepted / rejected / expired / countered
	parent_bid_id	uuid	self-FK, nullable – set when this bid is a counter-offer (bonus: negotiation thread)

Design note — bid expiry: I went with a **lazy expiration check** rather than a cron job: every read of a bid/request checks `deadline < now()` and treats it as expired in the response even if the stored status hasn't been flipped yet, and a lightweight scheduled job (every 5-15 min) does the actual status write so the DB stays consistent for anyone querying directly. Trade-off: a bid could show as "pending" in the DB for up to the cron interval after its real deadline, but no read-path ever returns a stale answer to a user, and I avoid the operational overhead of a precise per-bid timer. If exact-to-the-second expiry mattered here (it doesn't for this use case), I'd use a delayed job/queue with a TTL per bid instead.

3. ER Diagram

Core entities and relationships covering accounts, listings/requests, offers/bids, orders, and the fund-custody ledger, with key state fields called out on each entity.

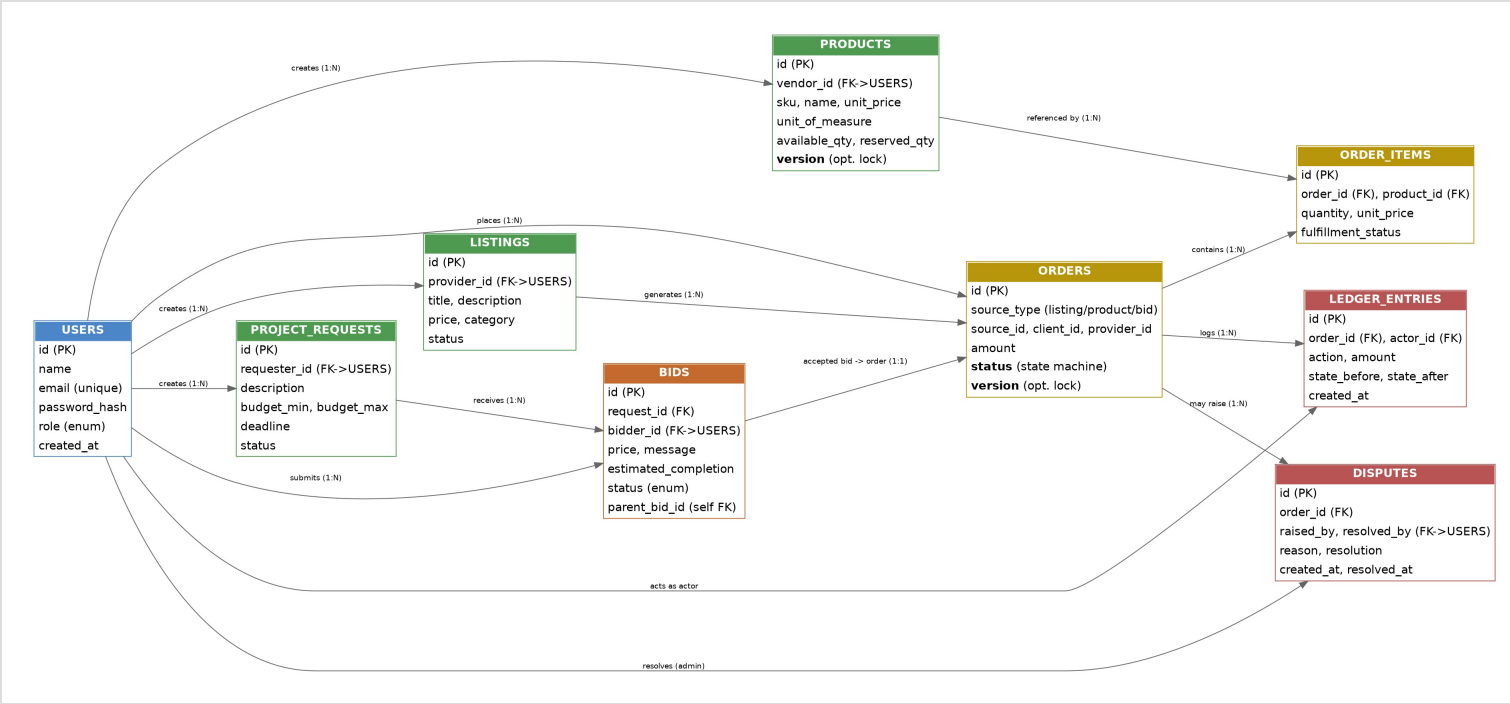


Fig. 2 — Entity-relationship diagram

4. API Structure

REST, JSON over HTTPS. All authenticated routes require a Bearer JWT; role checks happen server-side in middleware, never trusted from the client.

4.1 Auth & shared

POST	/auth/register
POST	/auth/login
POST	/auth/refresh
GET	/me
GET	/notifications
PATCH	/notifications/:id/read
GET	/messages/threads/:id
POST	/messages/threads/:id/messages

4.2 Module 1 — Service Marketplace

POST	/listings	GET	/listings?category=&price_min=&price_max=&q=
GET	/listings/:id		
POST	/listings/:id/request	// creates order in 'pending'	
POST	/orders/:id/accept	POST	/orders/:id/decline
POST	/orders/:id/fund	// simulated escrow hold	
POST	/orders/:id/deliver	// provider marks work delivered	
POST	/orders/:id/release	POST	/orders/:id/dispute
POST	/disputes/:id/resolve	// admin only	
GET	/orders/:id/ledger	// audit trail for one order	

4.3 Module 2 — Product Marketplace

POST	/products	GET	/products?category=&price_min=&in_stock=
POST	/cart/items	PATCH	/cart/items/:id
DELETE	/cart/items/:id		
POST	/orders	// multi-item, body: [{product_id, qty}, ...]	
POST	/orders/:id/items/:itemId/fulfill	// partial fulfillment	

4.4 Module 3 — Bidding System

POST	/requests	GET	/requests?status=
POST	/requests/:id/bids	GET	/requests/:id/bids?sort=price rating time
POST	/bids/:id/accept	// rejects siblings, creates linked order	
POST	/bids/:id/counter	// bonus: creates versioned child bid	

5. Authentication & Authorization Strategy

5.1 Authentication

Stateless JWTs: a short-lived access token (~15 min) plus a longer-lived refresh token stored as an httpOnly cookie. Passwords hashed with bcrypt (cost factor 12). I keep the access token short specifically because Module 1's escrow actions and Module 3's bid-accept are irreversible-ish state changes — I'd rather force a silent refresh than have a stale token still valid for hours.

5.2 Roles

Rather than a single fixed role column that would force one user into exactly one persona, I treat role as **contextual**: a user is a "Provider" with respect to a specific listing they created, a "Client" with respect to an order they placed, and so on. The `users.role` column stores a coarse default (used for signup flow / admin panel access), but every authorization check for a specific resource asks "is this user the owner/counterparty/admin of *this* record" rather than "does this user have role X globally." Concretely:

- **Provider** = `listings.provider_id == user.id`
- **Client** = `orders.client_id == user.id`
- **Vendor** = `products.vendor_id == user.id`
- **Requester** = `project_requests.requester_id == user.id`
- **Bidder** = `bids.bidder_id == user.id`
- **Admin** = `users.role == 'admin'` (this one *is* global, since dispute resolution needs to work across any order)

5.3 Enforcement

Authorization is implemented as a policy layer between the route handler and the service/repository — never in the frontend, and never scattered as inline if checks in controllers. Each domain action (accept order, release funds, resolve dispute, accept bid) maps to a small policy function that takes (user, resource) and returns allow/deny, e.g. `canReleaseFunds(user, order) = order.client_id === user.id && order.status === 'held'`. This keeps permission logic testable in isolation and out of the HTTP layer, and it's the same pattern I'd extend into Task 2's data-driven permission model (permissions as configuration per workflow template, rather than hardcoded per-route checks).

Why this matters here specifically: the spec calls out that "Admin role can resolve a dispute by releasing funds to either party" — that's the one place a non-owner needs write access to someone else's order, so it's handled as an explicit admin-only policy branch rather than a loophole in the regular client/provider checks.